

ЛАБОРАТОРНАЯ РАБОТА № 5

ЗНАКОМСТВО С ЯЗЫКОМ ГРАФИЧЕСКОГО ПРОГРАММИРОВАНИЯ. ПРОГРАММИРОВАНИЕ ВЫЧИСЛЕНИЙ

Цель работы: изучение пользовательского интерфейса *Visual LISP* в графической системе, способов ввода программы на *LISP*, способов программирования вычислений.

1 СПРАВОЧНЫЙ МАТЕРИАЛ

Графический язык программирования *Visual LISP* является расширением языка программирования *LISP*. *LISP* является языком высокого уровня, ориентированным на обработку списков, который выбран в качестве базового потому, что графические примитивы (начиная, с точки), блоки, наборы примитивов и блоков удобно представляются в виде списков.

Программа на языке *LISP* представляет собой построенный с помощью любого редактора текстов с соблюдением правил языка текстовый файл, который хранится на магнитном диске под определенным именем. Он может быть загружен в ОЗУ в ходе сеанса диалога в графической системе и может быть выполнен путем соответствующего ответа в командной строке:

COMMAND: (в русской версии – Команда:).

Программа представляет собой последовательность функций *Visual LISP* и функций, определенных самим программистом.

Помимо функций в тексте программы могут присутствовать комментарии. Комментарий – любой текст (он может быть и на русском языке), начинающийся со знака «;». Комментарием считается весь текст от знака «;» до конца строки. Комментарии не обрабатываются интерпретатором, а служат только для лучшего понимания текста программы.

Данные в *Visual LISP* принято называть *атомами*. Атомы – элементарные объекты, из которых формируются выражения *Visual LISP*.

Составной элемент программы на *Visual LISP*, состоящий из атомов или других списков, отделенных друг от друга пробелами, называется *списком*. Он всегда заключается в скобки, таким образом:

$\langle \text{список} \rangle ::= ([\langle \text{элемент} \rangle \dots])$, где $\langle \text{элемент} \rangle ::= \langle \text{атом} \rangle$.

Частным случаем списка является *функция*, определяемая следующим образом:

$\langle \text{функция} \rangle ::= (\langle \text{имя функции} \rangle [\langle \text{аргумент} \rangle \dots])$.

Здесь $\langle \text{имя функции} \rangle$ – символическое имя. Каждая функция имеет свое, связанное с ней имя.

Аргументы функции – выражения, построенные по правилам *Visual LISP*, в том числе другие функции. Для каждой конкретной функции характерны определенное количество, типы и порядок следования аргументов.

Все функции, которые можно использовать в своей программе, можно разделить на три группы:

первая – встроенные функции *Visual LISP*, т.е. те функции, имена которых и правила работы, с которыми изложены в соответствующем руководстве по *Visual LISP*;

вторая – функции, которые придуманы и описаны в своей программе пользователями или программистами;

третья – функции, описанные в других программах, которые можно подключить при выполнении своей программы.

Описание новой функции в программе должно предшествовать обращению к этой функции. Описание новой функции – тоже функция, имени которой предшествует ключевое слово *DEFUN*:

```
<описание функции>::=(DEFUN <имя функции>
  ([<аргумент>...] [/<локальная переменная>...])
  <выражение>).
```

Здесь <имя функции> есть имя, которое придумано для новой функции. Список аргументов есть список имен, обозначающих те данные, которые будут посылаться в функцию перед ее выполнением. Локальные переменные есть переменные, определение которых действует только внутри описываемой функции. Как аргументы, так и локальные переменные могут и отсутствовать, но скобки () после имени функции обязательны.

<Выражение> – любая запись, выполненная по правилам *Visual LISP*. Именно выражение показывает, какие действия должна выполнять описываемая функция.

Значение, возвращаемое функцией *DEFUN* есть имя описанной функции, т.е. после выполнения функции *DEFUN* можно в любом месте программы обращаться к описанной новой функции по ее имени. Примером использования функции *DEFUN* является приведенная ниже простая программа.

```
Программа 1. Вычисление суммы SIN(A) + COS(A)
(DEFUN SINCOS (A); имя функции и список аргументов
  (+ (SIN A) (COS A)); выражение: SIN(A) + COS(A)
) ;конец программы.
```

Программа 1 состоит из единственной функции *DEFUN*. Эта функция описывает придуманную нами функцию, которую мы назвали *SINCOS*.

Функция *SINCOS* имеет один аргумент, который мы обозначили именем *A*. Выражение в составе функции показывает, что должна делать данная функция.

Для **программирования вычислений** можно использовать следующие математические встроенные функции: + (сложение); - (вычитание); * (умножение); /(деление); *EXPT* (возведение в степень); *ABS* (определение модуля числа); *SIN* (синус); *COS* (косинус); *ATAN* (арктангенс); *SQRT* (квадратный корень); *EXP* (экспонента); *LOG* (натуральный логарифм) и некоторые другие.

Аргументы этих функций – целые или действительные числа. Углы выражаются в радианах. Функции «+», «-», «*», «/» могут иметь два аргумента и более, при этом у функции «-» первый аргумент обозначает

уменьшаемое, остальные аргументы – вычитаемое, у функции «/» первый аргумент – делимое, остальные аргументы – делители. Функция *EXPT* должна иметь два аргумента (первый – основание, второй – показатель степени), остальные функции должны иметь один аргумент.

Тип возвращаемого значения зависит от функции и типов аргументов. Функции *SIN*, *COS*, *ATAN*, *SORT*, *EXP*, *LOG* всегда возвращают действительные числа.

В графическую систему входит интерпретатор *LISP*, который начинает работать, если в командной строке в ответ на стандартную подсказку *Command:* (*Команда:*) ввести открывающую круглую скобку – «(». В этот момент графическая система настраивается на ввод и вычисление выражения, которое должно иметь форму списка, т.е. начинаться открывающей скобкой и заканчиваться закрывающей, а элементы списка должны отделяться друг от друга пробелами. Например, для получения суммы трех чисел можно ввести:

(+ 62.8 37.2 1923.5) .

Завершается ввод выражения нажатием клавиши <Enter> или пробела. После этого графическая система вычислит указанное выражение и выдаст результат: 2023.5.

В приведенном примере для вычисления было предложено выражение, открывающееся и закрывающееся круглыми скобками и состоящее из четырех элементов: первый – имя функции сложения (+), остальные – аргументы функции, т.е. слагаемые. Это иллюстрирует форму вычисляемых выражений языка графического программирования:

(<функция> [<аргумент1> [<аргумент2> ... [<аргументN>]]]) .

Здесь <функция> – это имя функции, <аргумент1>, <аргумент2>, ...<аргументN> – аргументы функции, разделяемые по крайней мере одним пробелом. Квадратные скобки в приведенной форме указывают на возможное отсутствие находящихся между ними аргументов. Количество аргументов зависит от синтаксиса функции. С точки зрения *LISP* все, что заключено в круглых скобках, является списком, который следует обработать как вычисляемое выражение. Основное правило *Visual LISP* – баланс скобок, т.е. количество открывающих скобок должно быть равно количеству открывающих.

Выражения *Visual LISP* могут быть простыми и составными. В составных выражениях могут использоваться другие выражения, причем разделяющими знаками в выражениях являются круглые скобки, пробелы и двойные кавычки. Двойные кавычки должны ограничивать с обеих сторон текстовые константы.

Ввод с клавиатуры выражений *LISP* не всегда удобен, поэтому можно записать предварительно свои строки на языке *LISP* в файл с расширением *lsp*, а затем загрузить этот файл с помощью функции *LOAD*. Синтаксис этой функции:

(LOAD <файл> [<ошибка>]) .

Здесь <файл> – строка с именем загружаемого файла а <ошибка> – выражение, которое нужно выполнить или значение, которое нужно вывести,

если загружаемый файл не будет обнаружен. Возвращаемое значение в случае удачной загрузки – результат последнего выражения или имя последней функции, если в файле находятся определения функций. В случае если загружаемый файл не найден, то выполняется <ошибка> (оно может быть текстом сообщения об ошибке). Аргумент <ошибка> может быть опущен. Аргумент <файл> обычно содержит имя файла, который ищется в соответствии с настройкой рабочего каталога и путей поиска файлов поддержки.

2 ВАРИАНТЫ ИНДИВИДУАЛЬНЫХ ЗАДАНИЙ ДЛЯ лабораторной работы № 5

- 1). Напишите и выполните программу для вычисления выражений $2AB$, $1/2AB$, $1/4*AB-AB$.
- 2). Поделите 1 на 50, затем прибавьте 0,8, вычислите тангенс полученного числа. Напишите и выполните программу для указанных вычислений.
- 3). Напишите и выполните программу для вычисления выражения $Y-B*X/Z+2*(B+A-C)$.
- 4). Напишите и выполните программу для вычисления выражения $(A+B*E)/(A+B+F)$.
- 5). Напишите и выполните программу для вычисления выражения $Y-B*X/Z+2*(B+A-C)$.
- 6). Напишите и выполните программу для вычисления выражения $1/5*(B*X/Z)+5*(B+A-80)$.
- 7). Напишите и выполните программу для вычисления выражения $44/(B-X/Z)+5*(B+A*80)$.
- 8). Напишите и выполните программу для вычисления выражения $1/5*(B+89)+5*(B+A-80)$.
- 9). Напишите и выполните программу для вычисления косинуса от выражения $1/5*(B*X/Z)+5*(B+A-8)$.
- 10). Напишите и выполните программу для вычисления синуса от выражения $10*(B/X/Z)+0,5*(B+A-80)$.
- 11). Напишите и выполните программу для вычисления выражения $1/6-A*C*D/(B*D/C)$.
- 12). Напишите и выполните программу для вычисления выражения $(23-67+138)-1/5*(B*X/Z)+5*(B+A-80)$.
- 13). Напишите и выполните программу для вычисления выражения $1/5*(B*X*Z)-5*(B+A-180)$.
- 14). Напишите и выполните программу для вычисления выражения $A*C+C/B-B/D-AD$.
- 15). Напишите и выполните программу для вычисления выражения $(d/b)*50/((a-c)*30)$.
- 16). Напишите и выполните программу для вычисления выражения $50-(B*30)+5*(B-80)$.

17). Напишите и выполните программу для вычисления выражения $1/5*(B*X/Z)+5*(B+A-80)$.

18). Напишите и выполните программу для вычисления котангенса от выражения $(B*X/Z)+5*(B+A-80)$.

19). Напишите и выполните программу для вычисления синуса от выражения $(B-X/Z)+5*(B+A-80)$.

20). Напишите и выполните программу для вычисления тангенса от выражения $5*(B+A-80)$.

21). Напишите и выполните программу для вычисления выражения $\frac{x}{y} + \frac{4x+b}{5y+g}$.

22). Напишите и выполните программу для вычисления тангенса от выражения

$$\frac{4x+b}{5y+g} \times \frac{1}{3x-y}$$

23). Напишите и выполните программу для вычисления синуса от выражения

$$\frac{1}{y^2} + \frac{4x \cdot b}{5y+g}$$

24). Напишите и выполните программу для вычисления выражения

$$\frac{x^2}{y} + \frac{4(x+b)}{5(y+g)}$$

25). Напишите и выполните программу для вычисления котангенса от выражения

$$\frac{z^3 \cdot x}{y} + \frac{x^4+b^4}{6/y+g}$$

26). Напишите и выполните программу для вычисления выражения

$$\frac{\sqrt{z}}{4x} + \frac{4x/b}{5y-g}$$

27). Напишите и выполните программу для вычисления выражения

$$\frac{\sqrt[3]{z} \times (6x+3)}{5y+g}$$

28). Напишите и выполните программу для вычисления синуса от выражения

$$\frac{2^x}{yz} + \frac{4x+b}{5y+g}$$

29). Напишите и выполните программу для вычисления выражения

$$\frac{4x+b}{5y+g} + \ln(z)$$

30). Напишите и выполните программу для вычисления выражения

$$\sin\left(\frac{x}{z}\right) + \frac{4x-b}{5y \cdot g}$$

31). Напишите и выполните программу для вычисления выражения

$$-2 \times \frac{x}{y} + \frac{x+b}{y+g}$$